



Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas Físico Químicas y Naturales

Departamento de Computación

Taller de Diseño de Software

Prof. Francisco Bavera

Compilador para el lenguaje C-TDS

“Entrega por etapas”

Emiliano Bernal

Agustín Cesari

Samuel Quintero

Octubre 2026

1. Decisiones de diseño: análisis léxico y sintáctico

1.1. Conflictos *shift/reduce* y *reduce/reduce* en `var_decl` y `method_decl`

La gramática inicial producía conflictos *shift/reduce* y *reduce/reduce* al definir `var_decl` y `method_decl`, ya que ambas producciones comparten el prefijo `type id` y el parser LALR(1) debe decidir, con un único token de anticipación (*lookahead*), qué regla aplicar.

Concretamente, la gramática inicial definía las declaraciones de variables y de métodos como dos listas recursivas separadas, una a continuación de la otra:

```

1 program: var_decls method_decls;
2
3 var_decls: %empty | var_decls var_decl;
4
5 method_decls: %empty | method_decls method_decl;
6
7 var_decl: type ID SEMI;
8
9 method_decl: type ID LPAREN params RPAREN block
10             | VOID ID LPAREN params RPAREN block
11             ;

```

Detectamos esta situación al intentar compilar `parser.y` con el siguiente ejemplo:

```

1 int main(){
2     int x;
3     int y;
4     x = 1;
5     y = 1;
6     x = x + 3 * 2 * y;
7     y = 2 + 2;
8     return;
9 }

```

1.1.1. Resolución del conflicto

Para resolverlo, se unificó la gramática en una única lista recursiva que admite `var_decl` y `method_decl` en cualquier orden:

```

1 program: %empty
2         | program var_decl
3         | program method_decl
4         ;

```

Esta unificación elimina la ambigüedad estructural: al no forzar un orden estricto entre declaraciones de variables y declaraciones de métodos, el parser deja de tener que decidir entre dos producciones que comienzan con el mismo prefijo en el mismo contexto, y los conflictos desaparecen.

1.1.2. Actualización del conflicto de la gramática

La anterior unificación, si bien solucionó el conflicto mencionado, propaga la aparición de elementos que no son declaraciones a la izquierda del árbol AST, lo cual se aleja de la especificación y complica innecesariamente el árbol. Decidimos en clase la siguiente modificación sobre la gramática.

```
1 program: var_decls method_decls
2 | method_decls
3 ;
4
5 var_decls: var_decls var_decl
6 | var_decl
7 ;
8
9 method_decls: method_decls method_decl
10 | method_decl
11 ;
```

De esta manera, solucionamos el conflicto manteniendo el orden de las variables declaradas y los métodos. Ahora se declaran todas las variables al principio y luego los métodos obligatoriamente.

2. Decisiones de diseño: análisis semántico

2.1. Decisiones, asunciones y aclaraciones

1. **El operador % solo admite operandos enteros.** La especificación solo exige que los operandos aritméticos sean `int` o `float`. Decidimos ser más estrictos con el módulo, porque su significado sobre números de punto flotante es ambiguo. Por lo tanto, `3.5 % 2` se informa como error semántico.
2. **El método main puede tener cualquier tipo de retorno.** La especificación exige que `main` exista y que no tenga parámetros, pero no restringe su tipo de retorno. Se chequean exactamente esas dos condiciones.
3. **Un método es visible dentro de su propio cuerpo.** El método se inserta en la tabla de símbolos *antes* de analizar su cuerpo, de modo que se admiten llamadas

recursivas.

4. **Parámetros y variables locales comparten un único scope.** Siguiendo la especificación, declarar en el cuerpo de un método una variable con el mismo nombre que un parámetro es un error de declaración duplicada.
5. **Los bloques anidados abren un scope propio y admiten *shadowing*.** Una variable declarada en un bloque interno puede tener el mismo nombre que una declarada en un scope exterior; la declaración interna oculta a la externa dentro del bloque.
6. **Variables y métodos comparten el espacio de nombres global.** No pueden existir una variable global y un método con el mismo nombre. Usar el nombre de un método como si fuera una variable también es un error.
7. **Coerción entre `int` y `float`.** Se permite asignar, pasar como argumento y retornar valores numéricos de un tipo donde se espera el otro (regla 14 de la especificación). El resultado de una operación aritmética es `float` si al menos un operando es `float`; en caso contrario es `int`. Elegimos `float` para no perder información en las operaciones mixtas.
8. **Resultado de operadores relacionales y lógicos.** Los operadores relacionales, la igualdad y los operadores lógicos siempre producen un valor `boolean`, aun cuando sus operandos sean inválidos. Esto evita errores derivados en expresiones que los contienen.
9. **El análisis no se detiene en el primer error.** Se informan todos los errores semánticos independientes, pero se evita repetir los que son consecuencia de uno ya informado (ver sección 2.3).
10. **Declaración antes del uso.** El análisis se realiza en una sola pasada y en orden textual. Un identificador debe estar declarado antes de su primer uso, tal como lo exige la especificación. En consecuencia, un método no puede invocar a otro definido más abajo en el archivo.

2.2. Compatibilidad de tipos y coerción

La relación de compatibilidad se centralizó en una función auxiliar:

```
1 static bool areCompatible(DataType expected, DataType received) {  
2     if (expected == TYPE_ERROR || received == TYPE_ERROR) {  
3         return true;  
4     }  
5     return expected == received
```

```

6      || (isNumeric(expected) && isNumeric(received));
7  }

```

Dos tipos son compatibles si son iguales o si ambos son numéricos. Esta única función se usa en asignaciones, argumentos de llamadas, expresiones de retorno e igualdad, de modo que la regla de coerción de la especificación queda definida en un solo lugar. El tipo resultante de cada operador es:

Operador	Operandos válidos	Resultado
+ - * /	int o float	float si alguno es float, si no int
%	solo int	int
<>	int o float	boolean
==	tipos compatibles	boolean
&&	boolean	boolean
!	boolean	boolean
- (unario)	int o float	mismo tipo del operando

2.3. Propagación de errores con TYPE_ERROR

Problema detectado. Al probar el analizador con la sentencia `x = z + 1;`, donde `z` no está declarada, se informaban varios errores por una única falta: primero que `z` no existe, luego que los operandos de `+` no son válidos y finalmente que la asignación mezcla tipos incompatibles. Los errores derivados no aportan información y ocultan la causa real. Lo detectamos con el caso de prueba `err_nodec1.ctds`, que debía producir un único error y producía dos.

Decisión. Se agregó el valor `TYPE_ERROR` al final de la enumeración `DataType` (para no alterar los valores existentes). Significa “en esta expresión ya se informó un error; continuar en silencio y propagarlo”. Cuando `checkExpr` encuentra una falta, la informa una única vez y devuelve `TYPE_ERROR`. Los nodos superiores que reciben ese tipo no informan nada y lo devuelven hacia arriba.

Para `x = z + 1;` el flujo es:

1. `checkExpr` sobre `z`: no está declarada, se informa el error y se devuelve `TYPE_ERROR`.
2. `checkExpr` sobre `+`: un operando es `TYPE_ERROR`, no se informa nada y se devuelve `TYPE_ERROR`.
3. `checkNode` sobre la asignación: `areCompatible` devuelve `true` si algún tipo es `TYPE_ERROR`, por lo que no se informa nada.

Resultado: un único mensaje, el de la causa.

Implementación. Tres piezas colaboran:

```
1  /* 1) areCompatible: si algun lado ya tiene error, no hay motivo nuevo */
2  if (expected == TYPE_ERROR || received == TYPE_ERROR) return true;
3
4  /* 2) operadores: no validan si algun operando ya fallo */
5  bool operandHasError = left == TYPE_ERROR || right == TYPE_ERROR;
6  if (!operandHasError && (left != TYPE_BOOL || right != TYPE_BOOL)) {
7      /* informar error */
8  }
9
10 /* 3) condiciones de if/while y operadores unarios */
11 if (condition != TYPE_ERROR && condition != TYPE_BOOL) {
12     /* informar error */
13 }
```

Notar que `areCompatible` devuelve `true` ante un `TYPE_ERROR` no porque los tipos sean correctos, sino porque no hay un motivo nuevo para informar. Es un `true` para continuar la propagacion y no de aprobación.

Limitación. El silenciamiento tiene un costo: si una misma expresión tiene dos faltas independientes en operandos distintos (por ejemplo, `z + true` con `z` sin declarar), solo se informa la primera; la segunda aparece al corregirla. Se priorizó que el compilador nunca repita errores derivados.